

YOLOv8 Object Detection and Tracking Application

This document provides a comprehensive explanation of the YOLOv8 Object Detection and Tracking Streamlit application, detailing its architecture, code flow, and functionality.

Table of Contents

- [1. Project Overview](#)
- [2. Project Structure](#)
- [3. Application Flow](#)
- [4. Authentication System](#)
- [5. Detection and Tracking Features](#)
- [6. Report Generation](#)
- [7. Technical Implementation Details](#)

Project Overview

This application provides a user-friendly web interface for performing object detection and tracking using YOLOv8 models. It allows users to upload images and videos, use their webcam, select specific objects to detect, and generate detailed reports of the results.

Key Features

- User authentication system
- Support for both object detection and segmentation
- Multiple input sources (image, video, webcam)
- Selective object detection
- Real-time object tracking
- Detection statistics with visualizations
- Report generation with downloadable PDF
- Responsive, user-friendly interface

Project Structure

The project consists of these essential files:

- app.py**: The main Streamlit application file that handles the user interface and orchestrates the entire application flow.
- helper.py**: Contains utility functions for detection, tracking, report generation, and other supporting operations.
- settings.py**: Stores configuration settings including model paths and source types.
- weights/yolov8n.pt**: The YOLOv8 model weights file (and optionally yolov8s-seg.pt for segmentation).

Application Flow

The application follows this general flow:

1. **Initialization:** The application loads, sets page configuration, and initializes session state.
2. **Authentication:** Users must log in with credentials to access the main application.
3. **Model Selection:** Users select whether to perform detection or segmentation.
4. **Source Selection:** Users choose between image, video, or webcam as the input source.
5. **Object Selection:** Users can select which specific objects they want to detect.
6. **Detection Process:** The application processes the input and performs detection/tracking.
7. **Results Display:** Detection results are displayed visually and summarized.
8. **Report Generation:** Users can save results and generate detailed reports.

Let's examine each of these stages in detail.

Authentication System

Implementation Details

The authentication system is implemented in the `check_password()` function in `app.py`. It uses Streamlit's session state to manage authentication status.

Flow:

1. When the application starts, it checks if the user is already authenticated via `st.session_state["password_correct"]`
2. If not authenticated, it displays a login form with username and password fields
3. When credentials are entered, the `credentials_entered()` function validates them against hardcoded values
4. If valid, the session state is updated and access is granted to the main application
5. A logout button in the sidebar allows users to log out, resetting the authentication state

Detection and Tracking Features

Image Detection Flow

1. **Upload:** User uploads an image via the file uploader in the sidebar
2. **Class Selection:** User selects which objects to detect (optional)
3. **Detection:** When the "Detect Objects" button is clicked:
 - The image is converted to a numpy array
 - The YOLOv8 model processes the image with the selected confidence threshold
 - Detection results are extracted and counted by class
 - The image with detection boxes is displayed
 - Results are displayed in a summary and detailed view
4. **Saving:** When the "Save Results" button is clicked:
 - The image with detection boxes is saved to a file
 - A PDF report is generated with detection statistics
 - Download links for both the image and report are displayed

Video Detection Flow

1. **Upload:** User uploads a video file via the file uploader
2. **Class Selection:** User selects which objects to detect (optional)
3. **Detection:** When the "Detect Video Objects" button is clicked:
 - The video is processed frame by frame with optimized frame skipping for performance

- ByteTrack tracking is applied to track objects between frames
 - A progress bar shows processing status
 - Detection results are updated in real-time with a line graph
 - Results are displayed under the video
4. **Saving:** When the "Save Results" button is clicked:
- A video file with detection boxes is created
 - A PDF report is generated with detection statistics and a line graph
 - Download links for both the video and report are displayed

Webcam Detection Flow

1. **Class Selection:** User selects which objects to detect (optional)
2. **Detection:** When the "Detect Objects (Webcam)" button is clicked:
 - The webcam is accessed with robust connection handling
 - Frames are processed at an optimized rate to maintain performance
 - ByteTrack tracking is applied to track objects between frames
 - Detection results are updated in real-time with a line graph
 - Results are displayed under the webcam feed
3. **Saving:** When the "Save Results (Webcam)" button is clicked:
 - A video file with recorded detection frames is created
 - A PDF report is generated with detection statistics and a line graph
 - Download links for both the video and report are displayed

Report Generation

The report generation functionality is implemented in the `create_detection_report()` function in `helper.py`. It creates a professional PDF document containing:

1. A title and generation timestamp
2. Summary statistics (total objects, unique object types)
3. A line graph visualization of detected objects
4. A detailed table of object counts

For images, the report includes the detection counts. For videos and webcam, the report includes a line graph showing how object counts changed over time.

Technical Implementation Details

YOLOv8 Integration

The application uses Ultralytics' YOLOv8 for object detection and tracking. The model is loaded once at startup and used for all detection operations.

```
# Loading the model
model = helper.load_model(model_path)

# Using the model for detection
res = model.predict(image_array, conf=confidence, classes=selected_classes)

# Using the model for tracking
res = model.track(image, conf=conf, persist=True, tracker="bytetrack.yaml", classes=selected_classes)
```

Performance Optimizations

Several techniques are used to optimize performance:

1. **Frame Skipping:** Only processing a subset of frames to maintain an acceptable frame rate:

```
process_every_n_frames = max(1, int(fps / target_fps))
if frame_index % process_every_n_frames == 0:
    # Process this frame
```

2. **Resolution Reduction:** Resizing large images and videos to a manageable size:

```
if w > MAX_WIDTH:
    ratio = MAX_WIDTH / w
    dim = (MAX_WIDTH, int(h * ratio))
    image = cv2.resize(image, dim, interpolation=cv2.INTER_AREA)
```

3. **Graph Update Limiting:** Updating the real-time graph less frequently:

```
if current_update_time - last_update_time > graph_update_interval:
    # Update the graph
```

4. **ByteTrack Integration:** Using ByteTrack for efficient object tracking between frames.

User Interface Design

The application uses Streamlit's layout features to create a clean, responsive interface:

1. The sidebar contains configuration options and input controls
2. The main area displays the detection results and visualizations
3. Expandable sections are used to show detailed results
4. Progress bars and status messages keep users informed during processing
5. Report download links are provided for saving results

Data Visualization

The application uses matplotlib for creating rich visualizations:

1. Line graphs in the application UI to show real-time detection counts
2. Power BI-style visualizations in the PDF reports

3. Detailed tables for showing object counts

Conclusion

This YOLOv8 Object Detection and Tracking application provides a powerful yet accessible interface for using state-of-the-art computer vision models. The architecture balances performance with user-friendliness, making advanced detection and tracking capabilities available through an intuitive web interface.

The modular code structure makes it easy to maintain and extend, while the optimized processing pipeline ensures good performance even on modest hardware.